

How the tests are organised and run

Every board owns its tests

How the tests are organised and run in aarch32 /

aarch64

Every board owns its tests.

That is the whole design, and everything below follows from it. There is no shared test directory, no list of test names anywhere in the build, and no capability filter: a board builds the directories it has, and changing a test reaches exactly one board.

1. Where they live

```
micro-os-plus-iii-aarch{32,64}/
├─ src/ include/                the ISA – no board, no test
├─ test/boards/<id>/
│   ├─ board.cmake              the board's facts, globbed
│   ├─ include/ src/            the board's silicon
│   ├─ hw.sh qemu.sh            how to run this board's tests
│   └─ test/                    ← this board's test applications
│       ├─ <app>/main.cpp        one directory per application
│       ├─ include/ src/         support shared by THIS board's tests
│       └─ tests.cmake           the knobs a directory listing cannot say
```

test/CMakeLists.txt in the port is a loop over test/\${BOARD}/*/. It names no test and no board.

Today

port	board	applications
aarch32	rpi-zero-2w	12
aarch32	rpi3b	12
aarch32	luckfox-lyra	13 — 11 shared-in-origin, plus smp_test_int and smp_test_int2 , which are this board's own
aarch64	rpi-zero-2w	12
aarch64	rpi3b	12

Each builds in two variants, so 24 targets per Pi board and 26 for the Lyra.

Why `rpi3b` has its own `test/` but not its own `src/`

`rpi3b` is the same silicon as `rpi-zero-2w`. Its `board.cmake` points `UOS_BOARD_SRC_DIR` at `../rpi-zero-2w`, so the two boards **share** `src/` and `include/`, compiled with `-DBOARD_RPI3B` — which is exactly how the predecessor's `make BOARD=rpi3b` worked, and that define reaches four places and no others:

file	what <code>BOARD_RPI3B</code> selects
<code>include/led.hpp</code>	a <i>different</i> LED driver — the ACT LED is not a SoC GPIO on this board but VideoCore expander GPIO 130, driven through the ARM↔VC mailbox property channel (tag <code>0x00038041</code>). 91 lines, against 36 for a plain GPIO.
<code>include/uart.hpp</code>	the banner strings
<code>include/smp.hpp</code>	<code>PORT_RAM_END</code> — the window a stack pointer is validated against
<code>src/mmu.cpp</code>	the top of the Normal-memory mapping

plus `linker-rpi3b.ld`, whose RAM region is that same number a third time. **Those last three are one fact written three times and must move together.** They did not, once: `PORT_RAM_END` lived unbranched in a header both boards share, so a Pi 3 B was linked and mapped for 1 GB while every thread stack above 512 MiB looked corrupt to the context switch.

The tests are *not* shared that way, because sharing them is what makes a change to one board able to break another. `_test_dir` is `test/${BOARD}`, never `${UOS_BOARD_SRC_DIR}/test` — which is exactly why `test/rpi3b/` exists beside `test/boards/rpi3b/board.cmake`.

The cost, stated plainly

The twelve applications exist in five copies across the two ports. A fix to a shared test — the `[OS_NCPU] = { 0, 0, 0, 0 }` kind — has to be applied in each copy. That is the trade: duplication bought in exchange for a board that cannot be broken from outside itself. The support code (`test/include/`, `test/src/`) is one copy *per board*, not per test, so the duplication is bounded by the number of boards.

2. `tests.cmake` — the three things a directory listing cannot say

Optional. A board whose tests need nothing beyond the defaults has no such file.

```

# Tests that reach the SD card or the USB device controller, and so link
# UOS_BOARD_DEVICES (board.cmake names which driver set that is).
set (BOARD_TEST_NEED_DEVICES
    sd_test smp-mat-sdcard-test smp-num-test smp-pipeline-test usb_test)

# Tests that bring their own start-up code and must NOT also get test/src/.
set (BOARD_TEST_SELF_CONTAINED smp_test_int smp_test_int2)

# Per-test -D flags, carried over from the predecessor's per-test Makefiles.
function (board_test_defines _app _out)
    if (_app STREQUAL "usb_test")
        set (${_out} LED_BLINKS=3 LED_ON_MS=40 LED_OFF_MS=40
            LED_GAP_MS=300 USB_FORCE_FS PARENT_SCOPE)
    endif ()
endfunction ()

```

`BOARD_TEST_NO_KERNEL` names the tests that compile **no kernel at all** — the bare-metal probes that run before a scheduler exists. They link the port's bare target (`UOS_PORT_BARE_LIB`) instead of its full one and are declared `NO_KERNEL`, so no kernel, no port scheduler and no `OS_NCPU` reach them. Only the Cortex-M port has such tests today; a port that declares no bare target rejects the request rather than quietly linking a kernel.

`BOARD_TEST_SELF_CONTAINED` exists because a test carried over whole from the predecessor defines its own `smp_install_boot_threads()`. Linking the board's shared copy as well is a duplicate definition, and picking one of them would mean editing a test that already passed on hardware.

Three more hooks answer "what if **one** test disagrees with its board", and each defaults to the board's own answer, so a board that does not use them never sees them:

```

# A linker script instead of the board's -- the RP2350B PSRAM tests link
# .data/.bss/heap into external memory at 0x11000000.
function (board_test_linker _app _out) ... endfunction ()

# How many CPUs ONE test runs on. uos_add_app() derives
# OS_USE_SMP_SCHEDULER from NCPU, so returning 1 on a two-core board is how a
# single-core test is expressed.
function (board_test_ncpu _app _out) ... endfunction ()

# Compile options on the test's OWN sources, not on the kernel it links --
# the RP2350 nested-interrupt tests need "-g0 -mlong-calls" because their
# handlers live in .data.
function (board_test_options _app _out) ... endfunction ()

```

`board_test_linker()` is also the per-test linker override that `smp-pro-cons-test` wanted here: it carries its own script in the predecessor, and until now the build had no way to say so.

Which tests a board has is not configured — it is observed

There is no `if (BOARD STREQUAL ...)` and no capability list gating tests. A board with no SD card has no `sd_test` directory. A single-CPU board will have no `smp_*` directories. This is what makes the `cortexm` port tractable, with six boards, half of them non-SMP.

3. The two variants

Every application is built once per variant, from the same sources:

variant	define	linker	for
<code><app>-qemu</code>	<code>QEMU_BUILD</code>	<code>UOS_BOARD_LINKER_QEMU</code>	the emulated suite
<code><app>-hwd</code>	<code>HW_BUILD</code>	<code>UOS_BOARD_LINKER_HW</code>	real silicon, over OpenOCD

This file describes the AArch32 and AArch64 ports. The Cortex-M port carries the same `test/CMakeLists.txt`, the same dispatchers and the same hooks — `board_test_defines()`, `board_test_sources()`, `board_test_includes()`, `board_test_linker()`, `board_test_ncpu()`, `board_test_options()`, `BOARD_TEST_SELF_CONTAINED` — so everything here about layout and per-test composition applies to it too. The last three hooks arrived *from* that port, where one board's tests disagree with their board about CPU count, memory map and compile options. What it does not have is a QEMU suite: all six of its boards are hardware-only. See `cortexm-port.md`.

The POSIX port carries the same loop and the same six hooks with one difference, and it is a difference in the machine rather than in the method: there is **one** variant, `host`, because the host *is* the machine — there is nothing to emulate and nothing to flash, and a test is simply an executable. It is deliberately not a `HW_BUILD`: that define means "use the board's real FAT32 boot partition", while the SD tests' other branch is flatfs over a plain image file, which is exactly what a host-file back-end is. Its dispatcher is `test/run.sh` rather than `qemu.sh` / `hw.sh`. See `posix-arch-port.md`.

How many variants is the board's decision, not this file's. A board that sets `UOS_BOARD_LINKER_QEMU` gets both; a board that leaves it unset gets `hwd` only, because there is nowhere for the other image to run. The absence *is* the declaration — there is no separate flag that could disagree with it.

The Luckfox Lyra is that case, and deliberately. QEMU has no RK3506 machine, and the generic `virt` models none of the blocks that make the board worth having — the DWC2 gadget, the DesignWare SD host, the SRAM mailbox, the Cortex-M0. An emulated Lyra would be a generic Cortex-A7 wearing the name. So the Lyra builds 19 images, not 38, and `test/qemu.sh` points at `hw.sh` when asked for it. The Pi boards build both variants and run the emulated suite.

`HW_BUILD` makes the SD tests use the board's existing FAT32 boot partition. `UOS_DEBUG_BOOT=ON` additionally compiles the early-boot assembly markers; it is off by default because each marker is a semihosting trap and under a JTAG probe each one costs real time.

4. Running them

Every board has the same two scripts, with the same interface. The port-level `test/hw.sh` and `test/qemu.sh` are dispatchers that contain no board names: `BOARD` picks the directory.

```
# emulated
BOARD=rpi3b test/qemu.sh           # the whole suite
BOARD=rpi3b test/qemu.sh smp_test0 # one test, output live

# hardware
BOARD=rpi3b test/hw.sh list        # what this build has
BOARD=rpi3b test/hw.sh smp_test0   # run one
BOARD=rpi3b test/hw.sh smp_test0 300 # ...with a 300 s budget

# the POSIX host (micro-os-plus-iii-posix-arch)
BOARD=native test/run.sh           # the whole suite
BOARD=native test/run.sh smp_test2 # one test, output live
```

or call the board directly — `test/boards/rpi3b/hw.sh smp_test0`. An unknown board gets the available ones listed rather than a wrong branch taken.

Each board's script holds only that board's facts and delegates the session itself to one shared runner in the kernel repository:

<code>test_smp1/run-qemu.sh</code>	runs every <code>*-qemu.bin</code> in a build directory, reports <code>PASS</code> / <code>SKIP</code> / <code>FAIL</code> from the <code>RESULT:</code> line each test prints, and seeds an SD image for the tests that need one
<code>test_smp1/run-host.sh</code>	the same, for the POSIX port: runs every <code>*-host</code> executable, same timeout table, same verdicts, same summary line — <code>run-qemu.sh</code> with the emulator taken out
<code>test_smp1/run-hw.sh</code>	halts the cores, enables semihosting, <code>load_image</code> , resumes

Those two files are the same for every board of every port. Nothing board-specific is in them and nothing generic is in the board scripts.

What a board script declares

```
UOS_HW_ENTRY=0x1003c          # where the image is linked
UOS_HW_SPIN_WORDS=4           # __smp_spin words to zero before release
UOS_HW_RESUME=cpsr            # cpsr | pc | entry
UOS_HW_NCPU=4
UOS_HW_TARGET_FMT="bcm2837.cpu%d"
UOS_HW_CFG=.../openocd-jlink-rpi3.cfg
UOS_HW_CFG_INIT=0             # 1 when the config is purely declarative
UOS_HW_PRELOAD="...TcL..."  # run between halt and load (the Lyra's
                                # MMU/cache sanitize)
```

The AArch64 Pi differs from the AArch32 Pi on the same silicon by four of these: entry `0x80000`, **eight** spin words (`__smp_spin` is `uint64_t[4]`), resume by PC because a core keeps the exception level it was halted in, and `CFG_INIT=1`.

5. Two rules the runners keep

Pure OpenOCD. No GDB anywhere — the configs set `gdb port disabled` outright. OpenOCD halts, loads and resumes; the application talks over the UART and semihosting.

No redirection. OpenOCD's output and the board's semihosting go to your terminal as they happen; the copy teed to `.hw-logs/<app>.log` exists only so the loop can match the verdict. The UART is never opened by any script, so `tio -b 115200 /dev/ttyACM0` in your own window is never fought over. A QEMU suite of a dozen tests is read from its summary, so those stay captured — but `qemu.sh <app>` streams, because that is the run you are watching.

One test per power cycle, on hardware. Every run is a `load_image` into RAM over whatever the previous test left there, and neither board has a reset a script can drive: the Pi has no SRST and its Cortex-A53 debug target has no reset method, and the Lyra's secondaries are released once, by clearing `__smp_spin`, which a second load cannot undo. `run-hw.sh` refuses a suite for this reason and takes exactly one test.

6. Adding a test, adding a board

A test, for one board: make `test/<board>/<name>/` with a `main.cpp`. That is all — it is globbed. If it needs the SD card, add its name to `BOARD_TEST_NEED_DEVICES`; if it brings its own start-up code, to `BOARD_TEST_SELF_CONTAINED`.

A test, for several boards: copy it into each board's `test/`. There is deliberately no mechanism to avoid this.

A board: make `test/boards/<id>/` with `board.cmake`, `include/`, `src/`, `hw.sh`, `qemu.sh`, a linker script and an OpenOCD config, and `test/<id>/` with its applications. Nothing outside those two directories is edited — not `CMakeLists.txt`, not `test/hw.sh`, not another board. `board.cmake` is globbed, and the configure step names any fact it forgot; `src/board-contract.cpp` compiles nothing and fails the build for a board that left `PORT_RAM_BASE`, `PORT_RAM_END`, `OS_NCPU`, `PORT_GREETING`, the banner strings, `SEMIHOST_TRAP_CHOSEN` or `OS_SMP_IPI_SGI` undeclared.

A board sharing another's silicon points `UOS_BOARD_SRC_DIR` at it and `#ifdef`s the differences, as `rpi3b` does — but it still gets its own `test/`.

7. Known gaps

- `smp-pro-cons-test` carries its **own** linker script in the predecessor (`smp-pro-cons-test/linker{,-rpi3b}.ld`). The build has no per-test linker override yet; it uses the board's.
- `test/luckfox-lyra/.pending/` holds six of the predecessor's Lyra tests that cannot link yet — three need FatFs compiled as C++ inside `namespace fatfs`, two need the RK3506 DWC2 device stack, one needs the Cortex-M0 firmware blob. A dot-directory is not globbed, so they are present without breaking the board. Its `README.md` names what each one needs. They are not to be rewritten.
- `smp_test4` has not been made to pass on the Lyra. The shared `smp_test4` is the Pi's no-affinity load-balancing test; the predecessor's Lyra `smp_test4` is a different program entirely, an 865-line CNTPCT benchmark.